

SOLID WALL CONSULTING

Agilité, DevOps & Sécurité

Scrum & Kanban · CI/CD, Docker, Kubernetes · Spring Security

Formateur : Haithem Mihoubi

Programme de la formation

- Module 1 — Gestion de projet Agile : Scrum & Kanban (3 jours)
- Module 2 — Culture & outils DevOps : CI/CD, Docker, Kubernetes (4 jours)
- Module 3 — Spring Security : JWT, OAuth2, Keycloak (5 jours)
- Fil rouge unique : l'application « QuickBite » (commande de repas)
- Pédagogie : théorie courte → démonstration → atelier → corrigé

Module 1 — Agilité

Scrum & Kanban, rôles PO / Scrum Master

Pourquoi l'Agilité ?

- Avant : cycle en cascade — tout spécifier, puis livrer 12-18 mois plus tard
- Conséquences : effet tunnel, besoins périmés, fonctions inutilisées
- Le coût du changement explose avec le temps
- Réponse : livrer par petits incréments et apprendre vite
- Idée clé : raccourcir la boucle entre une idée et sa validation utilisateur

Le Manifeste Agile (2001) — 4 valeurs

- Les individus et leurs interactions plutôt que les processus et outils
- Un logiciel qui fonctionne plutôt qu' une documentation exhaustive
- La collaboration avec le client plutôt que la négociation contractuelle
- L'adaptation au changement plutôt que le suivi d'un plan
- Les éléments de droite gardent de la valeur — on déplace le curseur

Les 12 principes (regroupés)

- Livrer de la valeur tôt et souvent, accueillir le changement
- Collaboration quotidienne métier ↔ développeurs, en face à face
- Mesure d'avancement = logiciel qui marche, à un rythme soutenable
- Excellence technique, simplicité, équipes auto-organisées
- S'améliorer régulièrement (rétrospective)

Prédictif vs Adaptatif

- Prédictif (cycle en V) : besoin stable, valeur à la fin, changement coûteux
- Adaptatif (Agile) : besoin évolutif, valeur à chaque itération
- Itératif $\neq$ incrémental — l'Agile combine les deux
- Métaphore : skateboard $\rightarrow$ trottinette $\rightarrow$ vélo $\rightarrow$ voiture

Concepts clés

- MVP : plus petite version livrable qui permet d'apprendre
- Valeur métier : on priorise par la valeur, pas par la facilité technique
- Boucle de feedback : construire → mesurer → apprendre
- Definition of Ready : prêt à être développé
- Definition of Done : réellement terminé (checklist non négociable)

Scrum — les fondations

- Cadre léger fondé sur l'empirisme
- 3 piliers : transparence, inspection, adaptation
- 5 valeurs : engagement, focus, ouverture, respect, courage
- Sprint = itération de durée fixe (1 à 4 semaines)

Scrum — les 3 rôles

- Product Owner : le QUOI et le POURQUOI ; possède le Product Backlog
- Scrum Master : leader serviteur ; facilite, lève les obstacles
- ≠ chef de projet : influence, pas autorité
- Développeurs : auto-organisés, pluridisciplinaires ; le COMMENT

Scrum — artefacts & événements

- Artefacts : Product Backlog, Sprint Backlog, Increment
- Sprint Planning (le plan) · Daily Scrum (15 min)
- Sprint Review (le produit, avec le client)
- Sprint Retrospective (la façon de travailler)

Écrire & estimer le travail

- Epic → User Story → Task
- Story : « En tant que ... je veux ... afin de ... »
- INVEST : Independent, Negotiable, Valuable, Estimable, Small, Testable
- Estimation en points (Fibonacci) + Planning Poker, pas en heures
- Priorisation MoSCoW : Must / Should / Could / Won't

Kanban

- Visualiser le travail sur un tableau (colonnes = étapes)
- Limiter le travail en cours (WIP) — « stop starting, start finishing »
- Métriques : lead time, cycle time, throughput
- Loi de Little : $\text{Lead time} \approx \text{WIP} \div \text{Throughput}$
- Réduire le WIP réduit mécaniquement le délai

Scrum vs Kanban vs ScrumBan

- Scrum : itérations fixes, rôles, Sprint Goal — développement produit
- Kanban : flux continu, pas de rôles imposés — flux imprévisible (support)
- ScrumBan : hybride — équipe Scrum noyée d'urgences, ou transition
- Pas de « meilleur » cadre : on choisit selon la nature du travail

Module 2 — DevOps

CI/CD, Docker, Kubernetes

DevOps, c'est quoi ?

- Briser le « mur » entre Dev (changement) et Ops (stabilité)
- Une culture + des pratiques, pas un métier ni un outil
- CAMS : Culture, Automation, Measurement, Sharing
- But : raccourcir le délai idée → production, de façon fiable

Mesurer : les métriques DORA

- Fréquence de déploiement
- Délai de livraison d'un changement (commit → prod)
- Taux d'échec des changements
- Temps moyen de rétablissement (MTTR)
- Vitesse et stabilité ne s'opposent pas

CI / CD / CD

- CI — Intégration Continue : tester à chaque commit
- CD — Livraison Continue : toujours déployable (déploiement manuel)
- CD — Déploiement Continu : déploiement automatique en prod
- Cycle DevOps : Plan → Code → Build → Test → Release → Deploy → Operate → Monitor

Git & intégration continue

- Branches courtes + Pull Requests + revue
- Conventional Commits : feat, fix, docs, refactor, test, chore
- Pipeline CI : checkout → install → lint → build → tests → artefact
- Règle d'or : on ne fusionne jamais une branche au pipeline rouge

Docker — conteneurisation

- Empaqueter l'app + son environnement → « marche partout »
- Conteneur : partage le noyau de l'hôte → léger, démarre en ms
- VM : embarque un OS complet → lourd
- Image (modèle) vs Conteneur (instance) ; volumes, réseaux, registres

Docker — Dockerfile & Compose

- Dockerfile multi-stage : build séparé du runtime → image légère
- Ordonner du moins au plus changeant (cache des couches)
- Ne pas tourner en root ; jamais de secret dans l'image
- Docker Compose : plusieurs services (app + base) en une commande

Kubernetes — orchestration

- Redémarrage auto, mise à l'échelle, déploiement sans coupure
- Pod (unité), Deployment (réplicas + mises à jour), Service (adresse stable)
- Ingress (accès HTTP externe), ConfigMap / Secret, Namespace
- Approche déclarative : on décrit l'état souhaité, K8s converge

Aller plus loin : Helm, observabilité, GitOps

- Helm : packager et paramétrer les manifestes (templates + values)
- Observabilité : logs, métriques (Prometheus), dashboards (Grafana)
- 4 signaux dorés : latence, trafic, erreurs, saturation
- GitOps (Argo CD) : Git = source de vérité, synchro automatique

Module 3 — Spring Security

Authentication, JWT, OAuth2, Keycloak

Authentification vs Autorisation

- AuthN : « Qui es-tu ? » (identité) — vient en premier
- AuthZ : « As-tu le droit ? » (permissions) — vient ensuite
- Authentifié ≠ autorisé : les deux contrôles sont obligatoires
- 401 = non authentifié · 403 = authentifié mais sans droit

Stateful vs Stateless & mots de passe

- Stateful : session côté serveur (cookie) — scale difficilement
- Stateless : token (JWT) — idéal API REST / microservices
- Mots de passe : jamais en clair → hachage lent + salé
- BCrypt / Argon2 (pas MD5/SHA seul, trop rapides)

Attaques courantes & parades

- Brute force → hachage lent, rate limiting, MFA
- MITM → HTTPS/TLS partout
- XSS → échapper les sorties, CSP, cookies HttpOnly
- CSRF → token anti-CSRF, SameSite, ou API stateless
- Injection SQL → requêtes paramétrées / ORM

Architecture Spring Security 6

- Chaîne de filtres (SecurityFilterChain) devant l'application
- AuthenticationManager → AuthenticationProvider → UserDetailsService
- SecurityContext : porte l'utilisateur authentifié
- WebSecurityConfigurerAdapter supprimé → bean SecurityFilterChain + lambdas

Autorisation RBAC

- Rôle (grossier : ADMIN) vs Permission (fin : order:read)
- Utilisateur → Rôles → Permissions
- @EnableMethodSecurity + @PreAuthorize("hasRole('ADMIN')")
- Piège : hasRole('ADMIN') attend l'autorité ROLE_ADMIN

JWT — JSON Web Token

- Auto-porteur et SIGNÉ → vérifiable sans appeler la base (stateless)
- 3 parties : header . payload . signature
- Signé ≠ chiffré : le payload est LISIBLE → aucune donnée sensible
- Access token court (15 min) + refresh token long (rotation)

OAuth2 & OpenID Connect

- OAuth2 : autorisation déléguée (accès en mon nom)
- OIDC : couche d'authentification (qui est l'utilisateur) + ID token
- Flow recommandé : Authorization Code + PKCE (web, mobile, SPA)
- ID token (pour le client) vs Access token (pour l'API)

Keycloak

- Serveur d'identité (IAM) : authentication, SSO, rôles, tokens
- Realm (espace isolé), Client (application), Roles, Mappers
- API Spring = resource server qui valide les tokens (issuer-uri)
- Mapper realm_access.roles → ROLE_* (cause n°1 des 403)

Durcissement & OWASP

- CORS (permission cross-domaine) ≠ CSRF (attaque)
- Rate limiting, gestion d'exceptions neutres, audit
- Microservices : API Gateway, rotation des clés de signature
- Risque n°1 OWASP : Broken Access Control → vérifier chaque accès

Projet fil rouge

QuickBite — une API qu'on construit, déploie et sécurise

QuickBite — atelier progressif

- A0-A1 : squelette Spring Boot + sécurité de base
- A2 : utilisateurs en base + RBAC (BCrypt, @PreAuthorize)
- A3 : authentication JWT (login / refresh / rotation)
- A4 : OAuth2 / Keycloak (optionnel)
- A5 : durcissement (CORS, exceptions, rate limit)
- DevOps : Docker → CI → Kubernetes

Synthèse du cursus

- Agile : construire LE BON produit, dans le bon ordre
- DevOps : le livrer VITE et SÛREMENT
- Sécurité : PROTÉGER ce qui est livré
- Trois facettes d'un même métier : livrer de la valeur, en confiance

Merci !

Questions / Réponses — Haithem Mihoubi